

Estrutura de Dados no Java

Aplicações Práticas de List vs Set

Objetivo

No mercado de trabalho, escolher a estrutura de dados incorreta pode causar instabilidades no sistema ou permitir inconsistência de dados (como duplicações indesejadas). A proposta deste exercício é aplicar `ArrayList` e `HashSet` em cenários arquiteturais práticos, garantindo a integridade das informações e o gerenciamento adequado em memória através do contrato `equals/hashCode`.

Contexto do Projeto (O Briefing)

A "TechCorp" está desenvolvendo um novo módulo para o seu sistema interno de Recursos Humanos e Controle de Acessos. O Arquiteto de Software responsável definiu as regras de negócio no Backlog, mas precisou focar em outras demandas. Você ficará responsável por codificar a infraestrutura desse fluxo.

O Backlog (Regras de Negócio e Implementação)

Crie um novo pacote no seu projeto chamado `br.com.techcorp.model` para as classes de negócio e `br.com.techcorp.main` para a execução principal.

RN01: O Funcionário e o Contrato do Hash (A Entidade)

O núcleo do sistema é o funcionário. No entanto, a TechCorp possui um desafio de consistência: existem muitos colaboradores com nomes idênticos na empresa.

- Crie a classe `Funcionario` com os atributos privados: `matricula` (String), `nome` (String) e `cargo` (String).
- Crie o construtor completo e os métodos getters.
- **A Regra de Identidade:** O sistema nunca deve se basear no nome do funcionário para identificá-lo de forma única. O único campo determinante é a `matricula`.
- **Ação:** Use a sua IDE para sobrescrever o `equals()` e o `hashCode()` da classe, selecionando **apenas a matrícula** como chave de comparação. Se esta etapa for omitida, as coleções otimizadas do Java poderão apresentar comportamentos inesperados.
- **Opcional:** Sobrescreva o `toString()` para imprimir os dados de forma estruturada e legível.

RN02: O Histórico de Entradas (A Catraca)

A portaria do prédio precisa registrar todas as vezes que um funcionário passa o crachá na catraca, mantendo a ordem cronológica exata. É normal (e esperado) que o mesmo funcionário passe na catraca várias vezes ao dia (entrada, almoço, volta, saída).

- Crie a classe `ControleDeAcesso`.
- Crie um atributo `historicoCatraca` que seja uma coleção dinâmica.
- **Dica de Arquitetura:** Qual coleção mantém a ordem e permite elementos repetidos? Instancie a coleção apropriada para este caso.
- Crie um método `registrarPassagem(Funcionario f)` que adicione o funcionário na referida coleção.

RN03: O Acesso ao Servidor (A Sala Segura)

A sala dos servidores é uma área de acesso restrito. Para evitar falhas de registro e acessos duplicados no sistema, a regra de negócio determina que: A mesma matrícula **NÃO PODE** estar registrada na sala mais de uma vez simultaneamente. A ordem de chegada não é o fator principal, apenas a exclusividade da matrícula.

- Na mesma classe `ControleDeAcesso`, crie um atributo `autorizadosSalaSegura`.
- **Dica de Arquitetura:** Qual coleção não garante ordenação, não permite itens duplicados e realiza verificações de forma otimizada? Instancie a coleção correta.
- Crie o método `concederAcessoSala(Funcionario f)`.
- **Atenção:** O método `.add()` da coleção escolhida retorna um boolean (true se conseguiu inserir, false se o elemento já estava presente). Use uma estrutura `if/else` baseada nesse retorno. Se for true, exiba: *"Acesso liberado para a Sala Segura"*. Se for false, exiba a seguinte notificação: *"Aviso: Matrícula já registrada na sala. Acesso não permitido."*

Validação do Sistema (A Classe Main)

Crie a classe executável `MainTechCorp` no pacote de execução para validar se a sua arquitetura previne registros duplicados indesejados de forma eficaz.

1. Instancie a classe `ControleDeAcesso`.
2. Instancie um objeto `Funcionario` padrão: f1 (Matrícula: "T-001", Nome: "Alice").
3. Instancie um objeto `Funcionario` duplicado: f2 (Matrícula: "T-001", Nome: "Alice Duplicada").

(Note que são dois objetos instanciados separadamente na memória, mas que representam a mesma matrícula lógica).

O Teste de Validação:

1. Tente registrar o f1 e depois o f2 na Catraca. Ambas as passagens devem ser registradas sem avisos (pois a catraca aceita histórico de múltiplas entradas).
2. Tente conceder o acesso à Sala Segura para o f1 e, logo em seguida, para o f2 (a representação duplicada).
3. **Validação Final:** O console DEVE imprimir a mensagem de "Aviso" na tentativa do f2. Se o sistema permitiu ambos os registros na Sala Segura, há uma inconsistência na implementação (provavelmente a RN01 não foi aplicada adequadamente ou a coleção incorreta foi escolhida na RN03).

Entrega e Versionamento

O console exibiu o aviso corretamente garantindo as restrições da sala? Excelente trabalho! É hora de versionar o seu código.

1. Limpe os alertas da IDE e salve todos os arquivos.
2. Abra o terminal (Git Bash) na raiz do projeto.
3. Rastreie as alterações realizadas:

```
git add .
```

4. Realize o commit com uma mensagem padronizada e descritiva:

```
git commit -m "feat: implementa estruturas List e Set no controle de acesso"
```

5. Envie para o repositório remoto:

```
git push origin main
```

(Copie a URL do seu repositório no GitHub e cole no formulário de entrega da disciplina).